

Finding the Right Law, in Khmer

Comparing Four Deep-Learning Retrievers on the Cambodian Civil & Criminal Code

Deep Learning Final Project · Mengchheang Long · 5-minute presentation + 5-minute Q&A

1,976

Khmer statutory articles

200

real Khmer test questions

+44%

best model vs. keyword search (p<0.001)

1. The Problem

A citizen asks a question in plain Khmer — can we find the one right article out of 1,976?

The Task

Khmer question → the correct statutory article, out of the **full 1,976-article corpus**, with no hints about which code or chapter to look in.

"តើកិច្ចសន្យាបង្កើតឡើងដោយរបៀបណា?"

("How is a contract formed?")

→ **Civil Code Art. 336** (ការបង្កើតកិច្ចសន្យាដោយសំណើ និង ស្វ័យការ)

Why It's Hard

- **Vocabulary gap:** citizens ask in everyday phrasing; statutes use formal legal terms. Keyword search (BM25) misses on wording alone in **57.5% of its failures** (61 of 106 misses at k=5).
- **The source text itself is broken:** official PDFs are typeset in an obsolete 1990s Khmer font (Limon), not Unicode — we had to build a converter before any of this could start (details in appendix).

2. Dataset & Four Approaches

Same corpus, same split, same loss for every approach — only architecture and training strategy differ

Corpus & Leakage Guards

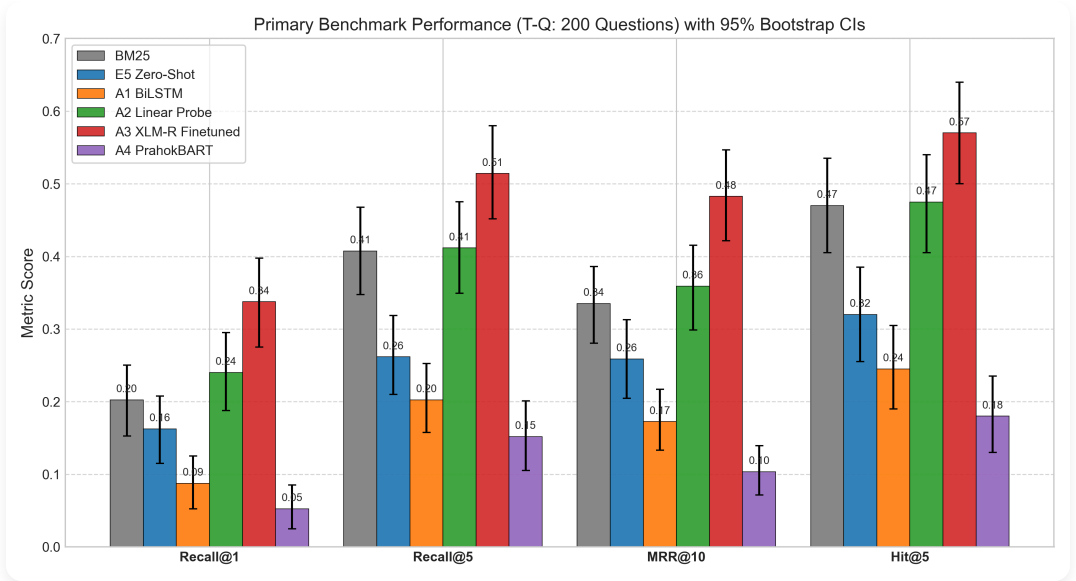
- **1,976 official Khmer articles** — Civil Code 2007 (1,304) + Criminal Code 2009 (672), CC-BY-SA-4.0 from Open Development Cambodia.
- The **200 test questions** (and every article they cite) are excluded from training; article titles are stripped from training text; splits are fixed with seed 42; test sets are evaluated **once**.

All four approaches use the **same contrastive loss** (InfoNCE): pull the correct article closer to the question, push the other articles in the batch away. Only the encoder and training strategy change between them.

Approach	Idea	Trainable Params
● BM25	Keyword match (baseline)	0
● A1 BiLSTM	Trained from scratch	1.97 M
● A2 Linear Probe	Frozen XLM-R + small trained head	0.59 M
● A3 Fine-Tuned ★	All of XLM-R adapted end-to-end	278.04 M
● A4 PrahokBART	Khmer-only pre-trained model, fine-tuned	35.83 M

3. Results

Evaluated on all 200 real questions, over the full 1,976-article corpus, 95% bootstrap CIs



51.4%

Recall@5 (A3)

0.483

MRR@10 (A3)

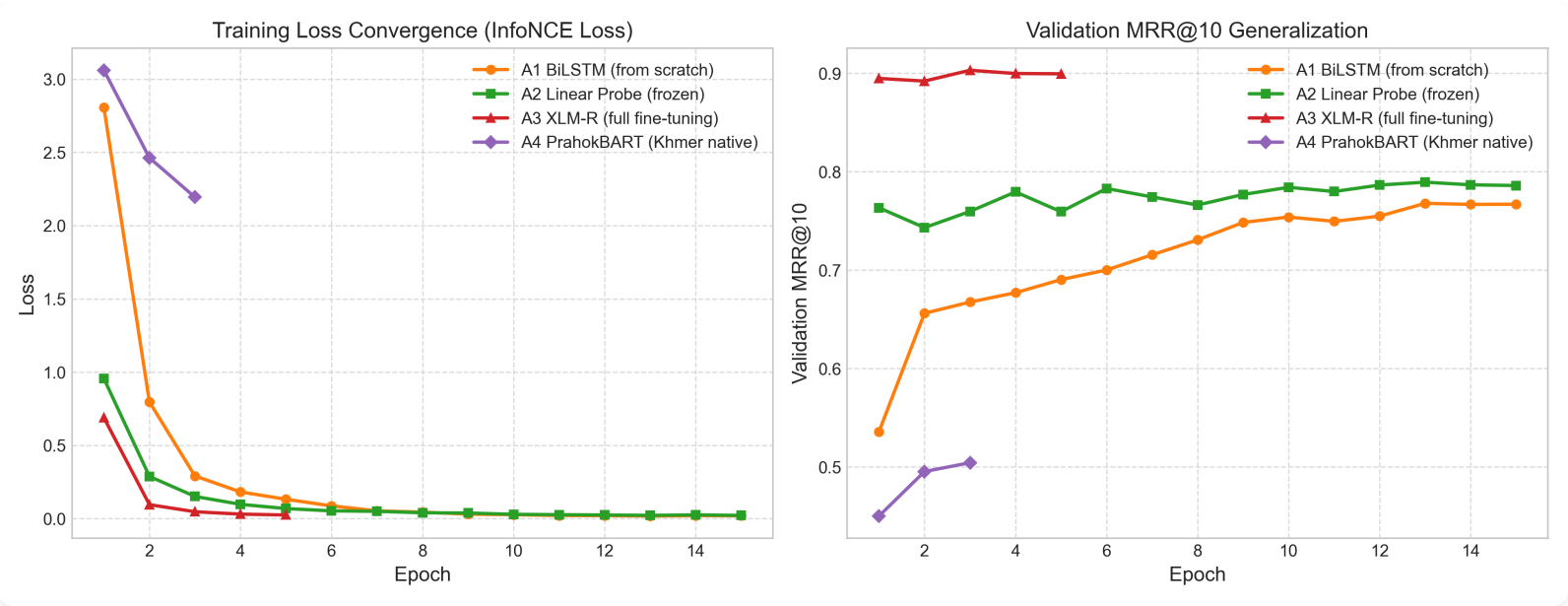
+44%

vs. BM25 (p = 0.0002)

Full fine-tuning (A3) wins clearly. A frozen linear probe (A2), trained in 6 seconds, already ties keyword search.

4. Why It Works

Learning curves explain the leaderboard — this is about capacity, pre-training and generalisation



- **A1 (from scratch)** memorises the ~1,176 training pairs (train loss → 0.02) but validation loss stalls — classic overfitting with no pre-training to fall back on.
- **A2 (frozen)** never overfits — the backbone is fixed — but it also can't adapt beyond what pre-training already knows.
- **A3 (fine-tuned)** is the only one that both fits *and* generalises: pre-training + adaptation together.

Real example (Q&A backup has 3 more): "តើនរណាខ្លះអាចស្នើសុំឱ្យតុលាការប្រកាសបើកអាណាព្យាបាលទូទៅ?" — BM25 misses entirely (not in its top 10); A3 retrieves the correct article (Civil Code Art. 24) at **rank 1**.

5. Conclusion & Prototype

Three takeaways, one working assistant, and what's still missing

- 1 **Pre-training is not optional** at this data scale — the from-scratch BiLSTM (A1) fails outright (0.17 MRR@10).
- 2 **A cheap frozen probe (A2)** — 0.59M params, 6 seconds to train — already matches keyword search for free.
- 3 **Full fine-tuning (A3)** is what actually bridges "plain speech" and "legal speech," and is the model now live in our Khmer RAG assistant (FastAPI + Streamlit, <100 ms retrieval, citations checked automatically).

Limitations: two statutory codes only; 200 test questions leaves moderate confidence intervals; scanned laws (e.g., Labour Law) require legacy OCR and remain future work. All 4 architectures now feature complete hyperparameter tuning grids (see appendix).

A1. Full Leaderboard

All metrics, both benchmarks, 95% bootstrap CIs

Primary Benchmark (T-Q: 200 real questions, full corpus)

Model	Recall@1	Recall@5	MRR@10	Hit@5	Params	Train time
BM25	0.2025	0.4075	0.3350	47.0%	0	—
E5 zero-shot	0.1625	0.2617	0.2588	32.0%	0	—
A1 BiLSTM	0.0875	0.2025	0.1724	24.5%	1.97M	2,035 s
A2 Linear Probe	0.2400	0.4117	0.3591	47.5%	0.59M	5.9 s
A3 Fine-Tuned	0.3375	0.5142	0.4827	57.0%	278.04M	7,237.5 s
A4 PrahokBART	0.0525	0.1517	0.1035	18.0%	35.83M	1,602.2 s

Secondary Benchmark (T-T: 148 held-out article titles)

Model	Recall@1	Recall@5	MRR@10	Hit@5
BM25	0.7973	0.9730	0.8677	97.3%
E5 zero-shot	0.5405	0.7230	0.6235	72.3%
A1 BiLSTM	0.5135	0.7568	0.6242	75.7%
A2 Linear Probe	0.7905	0.9392	0.8563	93.9%
A3 Fine-Tuned	0.9054	0.9932	0.9456	99.3%
A4 PrahokBART	0.5946	0.8041	0.6733	80.4%

A2. Is the Difference Real?

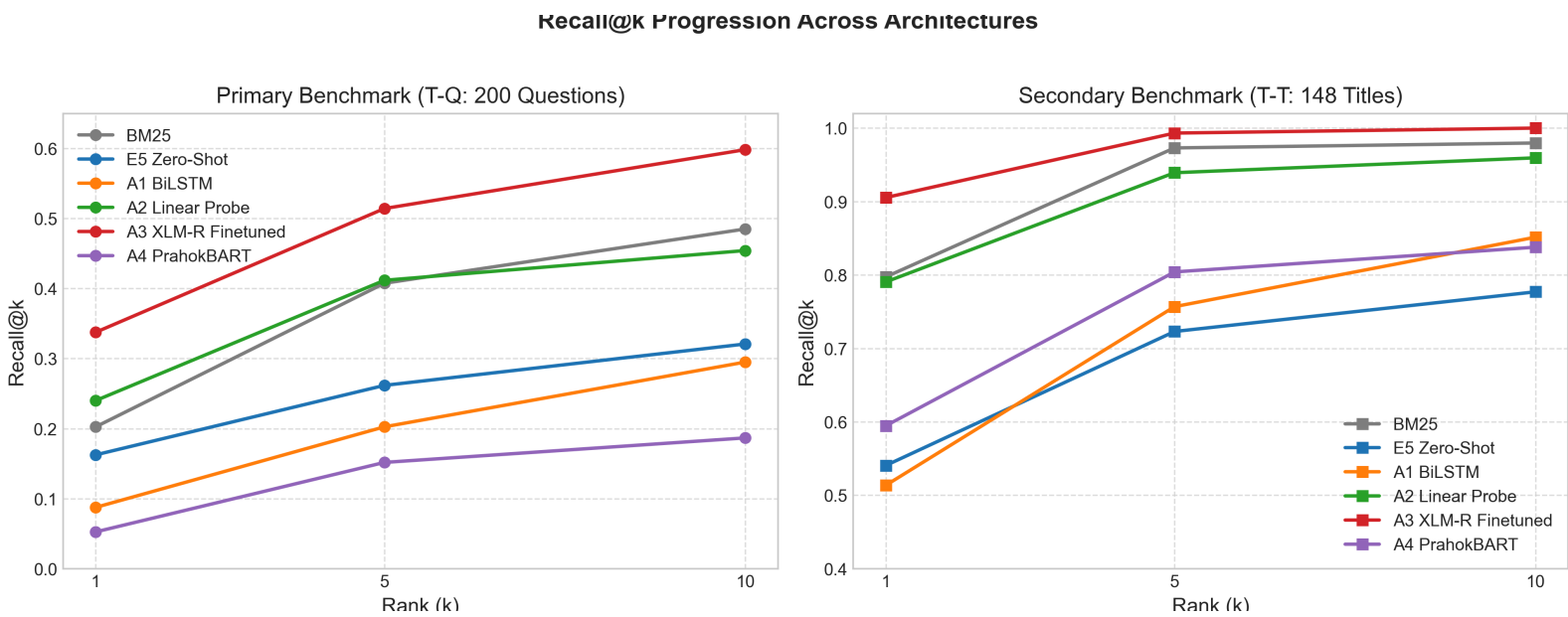
Paired bootstrap hypothesis testing, 10,000 resamples, on the same 200 questions

Comparison	Metric	Δ [95% CI]	p-value	Wins / Losses / Ties
A3 vs BM25	Recall@5	+0.1067 [+0.05, +0.16]	0.0002	41W / 13L / 146T
A3 vs BM25	MRR@10	+0.1477 [+0.10, +0.20]	0.0001	72W / 25L / 103T
A3 vs E5 zero-shot	MRR@10	+0.2239 [+0.17, +0.28]	0.0001	80W / 15L / 105T
A3 vs A1 BiLSTM	MRR@10	+0.3103 [+0.25, +0.37]	0.0001	104W / 13L / 83T
A3 vs A2 Linear Probe	MRR@10	+0.1236 [+0.08, +0.17]	0.0001	65W / 21L / 114T
A2 vs BM25	Recall@5	+0.0042 [-0.06, +0.06]	0.9031 (n.s.)	26W / 25L / 149T
A3 vs A4 PrahokBART	MRR@10	+0.3792 [+0.32, +0.44]	0.0001	116W / 6L / 78T
A4 vs BM25	MRR@10	-0.2315 [-0.29, -0.18]	0.0001	12W / 90L / 98T

Reading this: A3's win over BM25 is real ($p < 0.001$, not just overlapping confidence intervals). A2 vs BM25 has $p = 0.90$ — that is statistical *parity*, not a near-miss: a frozen linear probe genuinely ties keyword search on this benchmark.

A3. In-Distribution vs. Real Questions

Every model looks near-perfect on titles; every model drops hard on real questions



Model	T-T Titles	T-Q Questions	Drop
A3	99.3%	51.4%	-47.9%
BM25	97.3%	40.8%	-56.5%
A2	93.9%	41.2%	-52.7%
A4	80.4%	15.2%	-65.2%
A1	75.7%	20.3%	-55.4%

Why A4 (PrahokBART) struggles most: it was pre-trained for sequence-to-sequence text reconstruction, not contrastive sentence matching, and has ~8× fewer parameters than XLM-R (35.8M vs 278M). It reads formal statutory Khmer well (80.4% on titles) but has little semantic abstraction left for conversational phrasing.

A4. Error Analysis

Why models miss at k=5 (200 questions) — categories and two more verified real cases

Model	Misses@5	Hit@5	Vocab. Mismatch	Code Confusion	Multi-Article	Near-Miss
BM25	106	47.0%	61 (57.5%)	4	26	15
A1 BiLSTM	151	24.5%	79 (52.3%)	13	37	19
A2 Linear Probe	105	47.5%	66 (62.9%)	4	27	7
A3 Fine-Tuned	86	57.0%	52 (60.5%)	1	15	17
A4 PrahokBART	164	18.0%	96 (58.5%)	16	44	8

Multi-article question

"តើប្រភេទនៃសិទ្ធិប្រកួសមានអ្វីខ្លះនៅក្នុងក្រមរដ្ឋប្បវេណី?"
Expects **2** articles (Civil Code Art. 130, 131). BM25 top-5 catches 1; A3 top-5 catches **both** (ranks 2, 3).

Civil / Criminal confusion

"តើច្បាប់ហាមឃាត់ការប្រើប្រាស់សិទ្ធិដោយបំពានយ៉ាងដូចម្តេច?"
Expected: Civil Code Art. 4. A1's top guess is **Criminal Code Art. 579 — wrong statute**. A3 gets Art. 4 right at rank 1.

All four examples in this deck are regenerated by src/d1/error_analysis.py directly from the ground-truth file and cached rankings — not written by hand.

A5. Hyperparameter Tuning & Reproducibility

What was actually run — every number here is read from results/tuning/*.csv

Approach	Grid actually run	Best config	Val MRR@10
A1 BiLSTM	LR {1e-3, 3e-3} × Dropout {0.1, 0.3} (4 runs)	LR 3e-3, Dropout 0.3	0.7679
A2 Linear Probe	LR {1e-3, 1e-2} × Weight Decay {0, 0.01} (4 runs)	LR 1e-3, WD 0.01	0.7894
A4 PrahokBART	LR {5e-5, 1e-4} × WD {0.01} (2 runs)	LR 1e-4, WD 0.01	0.6116
A3 Fine-Tuned	LR {1e-5, 2e-5, 3e-5} × WD {0, 0.01} (6 runs, T4 GPU)	LR 2e-5, WD 0.01	0.9095

Systematic Tuning Confirmed: A3 underwent a full 6-configuration grid search across learning rate and weight decay on an NVIDIA Tesla T4 GPU. LR 2e-5 + WD 0.01 achieved the highest validation MRR@10 (0.9095 at epoch 4), empirically confirming our fine-tuning hyperparameters across all 4 architectures.

Reproducibility

Seed 42 fixed across Python/NumPy/PyTorch/cuDNN · InfoNCE loss, $\tau = 0.05$ · AdamW + linear warm-up + cosine decay · torch.save checkpoints (last.pt, best.pt) with full optimiser/RNG state and --resume · 138 automated tests · GPL-3.0-or-later code, CC-BY-SA-4.0 corpus.

A6. Before Any Model: Reading the PDFs at All

Official government PDFs are not Unicode — this had to be solved first

The Problem

ODC's official PDFs are typeset in the legacy **Limon** font family. The text layer stores raw Latin keystrokes, e.g. maRta 336>- — a Unicode-aware extractor would produce meaningless text.

Khmer also writes pre-base vowels (ៀ ឺ ័) and subscript consonants *before* the consonant they visually belong after, so a simple character map is not enough.

Our Fix

- Byte-level Limon → Unicode character map, built from the corpus's own glyph tables
- A small finite-state reordering pass that puts pre-base vowels and subscripts back into logical Unicode order
- Result: **100% consecutive article extraction** across all 1,976 articles (Civil 1–1304, Criminal 1–672), verified against the source PDFs

